

# Bander API Endpoints Definition

---



## API Endpoints

---



### User Endpoints

**POST**    `/user/register`

**Description:** Register a new user



**PDF Download**

### Example:

#### HTTP Request:

```
POST /user/register HTTP/1.1 Host: Bander Content-  
Type: application/json
```

#### Request Body:

```
{ "email": "user@example.com", "password":  
  "securepassword123" }
```

### Example Response (201 Created):

```
{ "token":  
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

### Possible Responses:

| Status Code | Description                                       |
|-------------|---------------------------------------------------|
| 201         | User registered successfully, returns JWT token   |
| 400         | Incorrect data entered or request body is missing |
| 409         | User with this email already exists               |
| 500         | Error creating user or server-side error          |

**POST**    /user/login

**Description:** Sign in user with email and password

### Example:

#### HTTP Request:

```
POST /user/login HTTP/1.1 Host: Bander Content-
Type: application/json
```

#### Request Body:

```
{ "email": "user@example.com", "password":
  "securepassword123" }
```

### Example Response (201 Created):

```
{ "token":
  "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

### Possible Responses:

| Status Code | Description                                       |
|-------------|---------------------------------------------------|
| <b>201</b>  | Sign in successful, returns JWT token             |
| <b>400</b>  | Incorrect data entered or request body is missing |
| <b>401</b>  | Incorrect email or password                       |
| <b>500</b>  | Error during sign in                              |

 **PDF Download**

**GET**    /user/:id

**Description:** Get user data by ID with band, instruments, and musical styles

### Example:

#### HTTP Request:

```
GET /user/1 HTTP/1.1 Host: Bander
```

#### Example Response (200 OK):

```
{ "id": 1, "username": "PopcornLover21", "email":  
  "user@example.com", "first_name": "John",  
  "last_name": "Doe", "city": "Budapest",  
  "birth_date": "1990-01-15", "is_admin": 0, "band":  
  { "id": 1, "name": "The Band" }, "instruments": [ {  
    "id": 1, "name": "Guitar" } ], "styles": [ { "id":  
    1, "name": "Rock" } ] }
```

#### Possible Responses:

| Status Code | Description                                           |
|-------------|-------------------------------------------------------|
| 200         | Successful query, returns user data with related info |
| 400         | Id is not valid                                       |
| 404         | User not found                                        |
| 500         | Error fetching user data                              |

 **PDF Download**

**GET**    /user/limit/:limit[/:offset]

**Description:** Get limited list of users with pagination

### Example:

#### HTTP Request:

GET /user/limit/10 HTTP/1.1 Host: Bander OR with  
offset: GET /user/limit/10/20 HTTP/1.1 Host: Bander

### Example Response (200 OK):

```
[ { "id": 1, "username": "user1", "email":  
  "user1@example.com", "first_name": "John" } ]
```

### Possible Responses:

| Status Code | Description                              |
|-------------|------------------------------------------|
| 200         | Successful query, returns array of users |
| 500         | Error fetching users                     |

 **PDF Download**

**PATCH**    /user/:id

**Description:** Update user data

**Example:**

**HTTP Request:**

```
PATCH /user/1 HTTP/1.1 Host: Bander Content-Type:
application/json
```

**Request Body:**

```
{ "username": "NewUsername", "first_name": "John",
  "last_name": "Doe" }
```

 **Possible Responses:**

| Status Code | Description               |
|-------------|---------------------------|
| 200         | User updated successfully |
| 404         | User not found            |
| 500         | Error updating user       |

 **PDF Download**

**POST**    /user/:id/profile-image

**Description:** Upload user profile image

**Example:**

**HTTP Request:**

```
POST /user/1/profile-image HTTP/1.1 Host: Bander
Content-Type: multipart/form-data file: [binary
image data]
```

 **Possible Responses:**

| Status Code | Description                 |
|-------------|-----------------------------|
| 200         | Image uploaded successfully |
| 404         | User not found              |
| 500         | Error uploading image       |

 **PDF Download**

**DELETE**    /user/:id

**Description:** Delete user by ID

**Example:**

**HTTP Request:**

```
DELETE /user/1 HTTP/1.1 Host: Bander
```

 **Possible Responses:**

| Status Code | Description               |
|-------------|---------------------------|
| 200         | User deleted successfully |
| 404         | User not found            |
| 500         | Error deleting user       |

 **PDF Download**





## Band Endpoints

**GET** `/band/limit/:limit[/:offset]`

**Description:** Get limited list of bands with pagination

### Example:

#### HTTP Request:

GET `/band/limit/10` HTTP/1.1 Host: Bander OR with  
offset: GET `/band/limit/10/20` HTTP/1.1 Host: Bander

### Example Response (200 OK):

```
[ { "id": 1, "name": "The Band", "city":  
  "Budapest", "profile_image_url": "https://  
example.com/band1.jpg", "banner_image_url":  
  "https://example.com/band1_banner.jpg",  
  "created_at": "2024-01-15T10:30:00Z" } ]
```



### Possible Responses:

| Status Code | Description                              |
|-------------|------------------------------------------|
| 200         | Successful query, returns array of bands |
| 500         | Error fetching bands                     |



**PDF Download**

GET /band/:id

**Description:** Get band data by ID

**Example:**

**HTTP Request:**

GET /band/1 HTTP/1.1 Host: Bander

**Example Response (200 OK):**

```
{ "id": 1, "name": "The Band", "city": "Budapest",  
  "profile_image_url": "https://example.com/  
band1.jpg", "banner_image_url": "https://  
example.com/band1_banner.jpg", "created_at":  
  "2024-01-15T10:30:00Z" }
```

 **Possible Responses:**

| Status Code | Description                         |
|-------------|-------------------------------------|
| 200         | Successful query, returns band data |
| 400         | Invalid band id                     |
| 404         | Band not found                      |
| 500         | Error fetching band                 |

 **PDF Download**

**POST**    /band/newband

**Description:** Create a new band

**Example:**

**HTTP Request:**

```
POST /band/newband HTTP/1.1 Host: Bander Content-
Type: application/json
```

**Request Body:**

```
{ "name": "New Band", "city": "Budapest" }
```

 **Possible Responses:**

| Status Code | Description               |
|-------------|---------------------------|
| 201         | Band created successfully |
| 400         | Invalid input data        |
| 500         | Error creating band       |

 **PDF Download**

**GET**    `/band/post/limit/:limit[/:offset]`

**Description:** Get limited list of latest band posts with pagination

**Example:**

**HTTP Request:**

`GET /band/post/limit/10 HTTP/1.1 Host: Bander`

**Example Response (200 OK):**

```
[ { "id": 1, "band_id": 1, "post_type":
  "announcement", "post_message": "New album coming
  soon!", "image_url": "https://example.com/
  post.jpg", "created_at": "2024-01-15T10:30:00Z",
  "band_name": "The Band" } ]
```

 **Possible Responses:**

| Status Code | Description                                   |
|-------------|-----------------------------------------------|
| 200         | Successful query, returns array of band posts |
| 500         | Error fetching band posts                     |

 **PDF Download**

**POST**    /band/post

**Description:** Create a new band post

### Example:

#### HTTP Request:

```
POST /band/post HTTP/1.1 Host: Bander Content-Type:
application/json
```

#### Request Body:

```
{ "band_id": 1, "post_type": "announcement",
  "post_message": "New album coming soon!",
  "expires_at": "2024-12-31T23:59:59Z" }
```

### Possible Responses:

| Status Code | Description                                   |
|-------------|-----------------------------------------------|
| 201         | Band post created successfully                |
| 400         | Invalid input data or missing required fields |
| 404         | Band not found                                |
| 500         | Error creating band post                      |

 **PDF Download**

## Genre Endpoints

**GET** /genre

**Description:** Get all genres/musical styles

### Example:

#### HTTP Request:

```
GET /genre HTTP/1.1 Host: Bander
```

#### Example Response (200 OK):

```
[ { "id": 1, "name": "Rock" }, { "id": 2, "name":  
  "Jazz" }, { "id": 3, "name": "Pop" } ]
```

### Possible Responses:

| Status Code | Description                               |
|-------------|-------------------------------------------|
| 200         | Successful query, returns array of genres |
| 500         | Error fetching genres                     |

 **PDF Download**

**GET**    /genre/:id

**Description:** Get genre by ID

**Example:**

**HTTP Request:**

GET /genre/1 HTTP/1.1 Host: Bander

**Example Response (200 OK):**

```
{ "id": 1, "name": "Rock" }
```

 **Possible Responses:**

| Status Code | Description                          |
|-------------|--------------------------------------|
| 200         | Successful query, returns genre data |
| 400         | Invalid genre id                     |
| 404         | Genre not found                      |
| 500         | Error fetching genre                 |

 **PDF Download**

**POST**    /genre

**Description:** Create a new genre (admin only)

**Example:**

**HTTP Request:**

```
POST /genre HTTP/1.1 Host: Bander Content-Type:
application/json Authorization: Bearer [JWT_TOKEN]
```

**Request Body:**

```
{ "name": "Blues" }
```

 **Possible Responses:**

| Status Code | Description                         |
|-------------|-------------------------------------|
| 201         | Genre created successfully          |
| 400         | Genre name is required              |
| 409         | Genre with this name already exists |
| 500         | Error creating genre                |

 **PDF Download**





## Instrument Endpoints

**GET** /instrument

**Description:** Get all instruments

### Example:

#### HTTP Request:

```
GET /instrument HTTP/1.1 Host: Bander
```

#### Example Response (200 OK):

```
[ { "id": 1, "name": "Guitar" }, { "id": 2, "name":  
"Piano" }, { "id": 3, "name": "Drums" } ]
```



### Possible Responses:

| Status Code | Description                                    |
|-------------|------------------------------------------------|
| 200         | Successful query, returns array of instruments |
| 500         | Error fetching instruments                     |



**PDF Download**

**POST**    /instrument

**Description:** Create a new instrument (admin only)

**Example:**

**HTTP Request:**

```
POST /instrument HTTP/1.1 Host: Bander Content-
Type: application/json Authorization: Bearer
[JWT_TOKEN]
```

**Request Body:**

```
{ "name": "Violin" }
```

 **Possible Responses:**

| Status Code | Description                              |
|-------------|------------------------------------------|
| 201         | Instrument created successfully          |
| 400         | Instrument name is required              |
| 409         | Instrument with this name already exists |
| 500         | Error creating instrument                |

 **PDF Download**

## Thread Endpoints

**GET**    /thread/:id/:numberofmessages

**Description:** Get thread with users and messages

### Example:

#### HTTP Request:

GET /thread/1/20 HTTP/1.1 Host: Bander

#### Example Response (200 OK):

```
{ "thread": { "id": 1, "created_at":
"2024-01-15T10:30:00Z" }, "users": [ { "id": 1,
"username": "user1" } ], "messages": [ { "id": 1,
"sender_id": 1, "message": "Hello!" } ] }
```

### Possible Responses:

| Status Code | Description                                              |
|-------------|----------------------------------------------------------|
| 200         | Successful query, returns thread with users and messages |
| 400         | Invalid thread id                                        |
| 404         | Thread not found                                         |
| 500         | Error fetching thread                                    |

 **PDF Download**

**POST**    /thread

**Description:** Create a new thread between two users

### Example:

#### HTTP Request:

```
POST /thread HTTP/1.1 Host: Bander Content-Type:
application/json
```

#### Request Body:

```
{ "user1_id": 1, "user2_id": 2 }
```

### Possible Responses:

| Status Code | Description                                    |
|-------------|------------------------------------------------|
| 200         | Thread already exists, returns existing thread |
| 201         | Thread created successfully                    |
| 400         | Invalid user ids or users must be different    |
| 404         | User not found                                 |
| 500         | Error creating thread                          |

 **PDF Download**



## Message Endpoints

**POST** /message

**Description:** Create a new message in a thread

### Example:

#### HTTP Request:

```
POST /message HTTP/1.1 Host: Bander Content-Type:
application/json
```

#### Request Body:

```
{ "thread_id": 1, "sender_id": 1, "message":
"Hello, this is my message!" }
```



### Possible Responses:

| Status Code | Description                              |
|-------------|------------------------------------------|
| 201         | Message created successfully             |
| 400         | Invalid input or missing required fields |
| 403         | Sender is not in this thread             |
| 404         | Thread or sender not found               |
| 500         | Error creating message                   |



**PDF Download**

**DELETE**    `/message/:id`

**Description:** Delete a message by ID

**Example:**

**HTTP Request:**

```
DELETE /message/1 HTTP/1.1 Host: Bander
```

 **Possible Responses:**

| Status Code | Description                  |
|-------------|------------------------------|
| 200         | Message deleted successfully |
| 400         | Invalid message id           |
| 404         | Message not found            |
| 500         | Error deleting message       |

 **PDF Download**

## Report Endpoints

**GET** `/report`

**Description:** Get all reports (admin only)

### Example:

#### HTTP Request:

```
GET /report HTTP/1.1 Host: Bander Authorization:
Bearer [ADMIN_JWT_TOKEN]
```

#### Example Response (200 OK):

```
[ { "id": 1, "reporter_id": 1, "reported_user_id":
2, "report_status": "open", "report_message": "User
is spamming", "created_at": "2024-01-15T10:30:00Z"
} ]
```

#### Possible Responses:

| Status Code | Description                                |
|-------------|--------------------------------------------|
| 200         | Successful query, returns array of reports |
| 500         | Error fetching reports                     |

 **PDF Download**

**GET** /report/:id

**Description:** Get report by ID

**Example:**

**HTTP Request:**

```
GET /report/1 HTTP/1.1 Host: Bander
```

 **Possible Responses:**

| Status Code | Description                           |
|-------------|---------------------------------------|
| 200         | Successful query, returns report data |
| 400         | Invalid report id                     |
| 404         | Report not found                      |
| 500         | Error fetching report                 |

 **PDF Download**



**POST**    /report

**Description:** Create a new report

### Example:

#### HTTP Request:

```
POST /report HTTP/1.1 Host: Bander Content-Type:
application/json
```

#### Request Body:

```
{ "reporter_id": 1, "reported_user_id": 2,
  "report_message": "User is spamming or violating
rules" }
```

### Possible Responses:

| Status Code | Description                              |
|-------------|------------------------------------------|
| 201         | Report created successfully              |
| 400         | Invalid input or missing required fields |
| 404         | Reporter or reported entity not found    |
| 500         | Error creating report                    |

 **PDF Download**

**DELETE**    /report/:id

**Description:** Delete a report by ID (admin only)

**Example:**

**HTTP Request:**

```
DELETE /report/1 HTTP/1.1 Host: Bander
Authorization: Bearer [ADMIN_JWT_TOKEN]
```

 **Possible Responses:**

| Status Code | Description                 |
|-------------|-----------------------------|
| 200         | Report deleted successfully |
| 400         | Invalid report id           |
| 404         | Report not found            |
| 500         | Error deleting report       |

 **PDF Download**

**PATCH**    /report/:id

**Description:** Update report status (admin only)

**Example:**

**HTTP Request:**

```
PATCH /report/1 HTTP/1.1 Host: Bander Content-Type:
application/json Authorization: Bearer
[ADMIN_JWT_TOKEN]
```

**Request Body:**

```
{ "report_status": "resolved" }
```

 **Possible Responses:**

| Status Code | Description                        |
|-------------|------------------------------------|
| 200         | Report status updated successfully |
| 400         | Invalid status or input            |
| 404         | Report not found                   |
| 500         | Error updating report              |

 **PDF Download**



# Authentication

---

## JWT Token

**Type:** Bearer Token

**Location:** Authorization Header

**Expiration:** 2 hours

```
Authorization: Bearer  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```



**PDF Download**